

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Многопоточность

В операционной системе могут одновременно работать множество процессов. При запуске любой программы запустится новый процесс.

Поток является неким элементом в каком-то процессе. В одном процессе может быть несколько потоков, которые используют совместно одни и те же ресурсы.

Потоки создаются в одном главном потоке **GIL (Global Interpreter Lock)**. И GIL гарантирует что только один процессор будет работать для запущенного скрипта пайтон. То есть такой подход имеет преимущества для однопоточных программ, помогая избегать кофликтов. GIL руководствуется следующим правилом - любое выполнение байткода в Python требует блокировки интерпретатора.

GIL нужен для решения проблемы **race conditions** – состояние гонки. У Python автоматическое управление памятью, и если у нас запущено 2 потока, которые используют один и тот же объект, то когда первый поток обращается к какому-либо участку памяти, то второй поток блокируется при попытке получить доступ.

Модуль **threading**

Модуль **threading** значительно упрощает работу с потоками и позволяет нам запрограммировать запуск нескольких операций одновременно. Модуль содержит класс **Thread**. Экземпляры класса **threading.Thread** представляют потоки.

target - объект, который будет вызван

name - имя потока

args или **kwargs** - передача позиционных или именованных параметров соответственно, для вызываемого заданного в параметре **target** объекта. В примере ниже аргумент передан в списке, но он может быть и строкой, и кортежем.

Метод **start()** запускает поток.

```
import threading

def func(n):
    print("Поток", n)

p1 = threading.Thread(target=func, name="t1", args=[1])
p2 = threading.Thread(target=func, name="t2", args=[2])
p1.start()
p2.start()
```

my_thread x

Поток 1

Поток 2

Этот же пример с использованием именованных параметров, и выводом имени потока.

```
import threading

def func(n):
    print("Поток", n)
    print(threading.current_thread().getName())

p1 = threading.Thread(target=func, name="t1", kwargs={"n": "1"})
p2 = threading.Thread(target=func, name="t2", kwargs={"n": "2"})
p1.start()
p2.start()
```

my_thread x

```
Поток 1
t1
Поток 2
t2
```

Метод `currentThread()` показывает имя и идентификатор потока, а если к нему применить `getName()`, то извлекается имя.

my_thread x

```
Поток 1
<Thread(t1, started 140494499735296)>
Поток 2
<Thread(t2, started 140494420047616)>
```

Если же мы не указали в аргументах имена потоков, но мы их выводим с помощью методов, то они на выходе будут иметь стандартное имя Thread.

my_thread x

```
Поток 1
<Thread(Thread-1, started 140625376835328)>
Поток 2
<Thread(Thread-2, started 140625368442624)>
```

my_thread x

```
Поток 1
Thread-1
Поток 2
Thread-2
```

Метод `join()` останавливает поток, когда тот выполнит свои задачи. Завершение потоков явным образом не позволяет работать главному потоку параллельно. То есть он будет вынужден ждать завершения других потоков. В скобках мы можем указать в секундах время блокировки главного потока перед продолжением его работы.

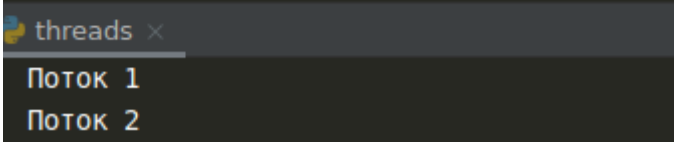
Мы также можем написать отдельный класс, который наследуется от `Thread` из модуля `threading`. В этом случае программу действий потока прописывают в методе `run()` созданного класса.

```
import threading

class MyThread(threading.Thread):
    def __init__(self, num):
        super().__init__(name="t" + num)
        self.num = num

    def run(self):
        print("Поток", self.num)

thread1 = MyThread("1")
thread2 = MyThread("2")
thread1.start()
thread2.start()
thread1.join()
thread2.join()
```



Модуль `threading` включает в себя один очень удобный класс `Timer`, который вы можете использовать для запуска действия спустя определенный отрезок времени. Данный класс запускает собственный поток и начинают работу с того же метода `start()`, как и обычные потоки.

На картинке ниже сначала выводится сообщение `"Привет! Через 10 секунд попрощаемся )"` – это сообщение выводится в главном потоке. И спустя 10 секунд появляется сообщение `'До свидания!'`. Таким образом мы можем запускать какую-либо функцию спустя определённый промежуток времени, указанный в качестве параметра.

```
import threading

def sec():
    print('До свидания!')

t = threading.Timer(10, sec)
t.start()
print("Привет! Через 10 секунд попрощаемся ")
```

```
reg x
Привет! Через 10 секунд попрощаемся )
До свидания!
```

Если для потока мы применим метод `join()`, то главный поток будет ждать 10 секунд, и выведет только после этого сообщение "Пока! Пришлось тебя ждать."

```
import threading

def sec():
    print('До свидания!')

t = threading.Timer(10, sec)
t.start()
t.join()
print("Пока! Пришлось тебя ждать.")
```

```
reg x
До свидания!
Пока! Пришлось тебя ждать.
```

А если установить задержку, то основной поток напечатает сообщения спустя время задержки. Метод `join()` можно использовать много раз, и писать число класса `float`, то есть с плавающей точкой.

```
t = threading.Timer(10, sec)
t.start()
t.join(2)
print("Пока! Пришлось тебя ждать.")
```

```
reg x
Пока! Пришлось тебя ждать.
До свидания!
```


А теперь рассмотрим преимущества многопоточности на примере. Сначала накидаем код без потоков. В коде мы запустим по-очереди в цикле 3 стрелы, которые поразят цели спустя 5 секунд полёта.

```
import time

start = time.time()
def sec():
    print('Стрела полетела...')
    time.sleep(5)
    print('Стрела поразила цель!')

for i in range(3):
    sec()
print(f'Completed in: {(time.time() - start):.3f} sec.')
```

```
reg x
Стрела полетела...
Стрела поразила цель!
Стрела полетела...
Стрела поразила цель!
Стрела полетела...
Стрела поразила цель!
Completed in: 15.009 sec.
```

Как видно, общее время у нас 15 секунд, потому что стрелы запускались по-очереди. Теперь изменим код и добавим в него потоки.

```
import threading
import time

start = time.time()
def sec(x):
    print('Стрела полетела...')
    time.sleep(x)
    print('Стрела поразила цель!')

my_list = []
for i in range(3):
    my_thread = threading.Thread(target=sec, args=(5,))
    my_list.append(my_thread)
    my_thread.start()

for my_thread in my_list:
    my_thread.join()
print(f'Completed in: {(time.time() - start):.3f} sec.')
```

В цикле создаются потоки по количеству итераций в цикле. Каждый поток мы добавляем в список, который также прогоняем в цикле. Результат сильно отличается от предыдущего, всего 5 секунд против 15-ти.

```
Стрела полетела...
Стрела полетела...
Стрела полетела...
Стрела поразила цель!
Стрела поразила цель!
Стрела поразила цель!
Completed in: 5.005 sec.
```

Функция `threading.active_count()` возвращает количество потоков, которые на данный момент живы. Этот код демонстрирует работу данной функции. Можно писать и как на скриншоте ниже `threading.activeCount()`.

```
import threading
import time

def proc(n, s):
    time.sleep(s)
    print('Поток', str(n), 'завершился')

for x in range(1, 4):
    print('Поток', str(x), 'стартовал')
    print('Количество активных потоков ', threading.activeCount())

    if x == 1:
        threading.Thread(target=proc, args=[x, 2]).start()
    elif x == 2:
        threading.Thread(target=proc, args=[x, 3]).start()
    elif x == 3:
        threading.Thread(target=proc, args=[x, 1]).start()
```

```
threads x
Поток 1 стартовал
Количество активных потоков 1
Поток 2 стартовал
Количество активных потоков 2
Поток 3 стартовал
Количество активных потоков 3
Поток 3 завершился
Поток 1 завершился
Поток 2 завершился
```

Lock – блокировка для синхронизации потоков

Чтобы не произошла ситуация, когда два потока пытаются одновременно обратиться к одному объекту применяются замки. Замок останавливает другие потоки, пытающиеся попасть в ту же область программы.

Простейший замок может быть реализован на основе класса **Lock** модуля **threading**. Замок имеет два состояния: он может быть или открыт, или заперт. В последнем случае им владеет некоторый поток.

Объект класса **Lock** имеет следующие методы:

- **acquire([blocking=True])** Делает запрос на записание замка. Если параметр **blocking** не указан или является истиной, то поток будет ожидать освобождения замка. Если параметр не был задан, метод не возвратит значения. Если **blocking** был задан и истинен, метод возвратит **True** (после успешного овладения замком). Если блокировка не требуется (то есть задан **blocking=False**), метод вернет **True**, если замок не был заперт и им успешно овладел данный поток. В противном случае будет возвращено **False**.
- **release()** Запрос на отпирание замка.
- **locked()** Возвращает текущее состояние замка (**True** - заперт, **False** - открыт). Следует иметь в виду, что даже если состояние замка только что проверено, это не означает, что он сохранит это состояние до следующей команды.
- **threading.RLock** Отличается от **threading.Lock** тем, что некоторый поток может запрашивать его записание много раз. Отпирание такого замка должно происходить столько же раз, сколько было запираций.

Напишем пример когда 2 потока одновременно обращаются к одному ресурсу. В данном случае глобальной переменной **x**. В коде 100000 итераций, где к нулю прибавляется единица. А так как потока два, то на выходе мы должны получить 200000. Однако результаты отличаются для разных итераций.

```
Iteration 1: x = 152886
Iteration 2: x = 200000
Iteration 3: x = 200000
Iteration 4: x = 200000
Iteration 5: x = 200000
Iteration 6: x = 200000
Iteration 7: x = 200000
Iteration 8: x = 200000
Iteration 9: x = 198331
Iteration 10: x = 200000
```

```
Process finished with exit code 0
```

В некоторых случаях результат неверный, так как переменная была занята одним потоком, и второй не смог выполнить в это время операцию. Код программы:


```

import threading

x = 0

def increment():
    global x
    x += 1

def thread_task():
    for _ in range(100000):
        increment()

def main_task():
    global x
    x = 0
    t1 = threading.Thread(target=thread_task)
    t2 = threading.Thread(target=thread_task)
    t1.start()
    t2.start()
    t1.join()
    t2.join()

for i in range(1, 11):
    main_task()
    print("Iteration {0:>2}: x = {1}".format(i, x))

```

Всего лишь три строчки кода с замками могут легко решить эту проблему.

```

def thread_task():
    lock.acquire()
    for _ in range(100000):
        increment()
    lock.release()

def main_task():
    global x
    x = 0
    t1 = threading.Thread(target=thread_task)
    t2 = threading.Thread(target=thread_task)
    t1.start()
    t2.start()
    t1.join()
    t2.join()

lock = threading.Lock()
for i in range(1, 11):

```

Замки также поддерживают контекстного менеджера `with`, что позволяет писать код ещё проще. Методы `acquire()` и `release()` писать в таком случае вовсе не требуется.

```
def thread task():
    with lock:
        for _ in range(100000):
            increment()
```

Однако при работе с замками может возникнуть тупиковая ситуация (**deadlock**) – возникает из-за того, что двум потокам потребуются одни и те же ресурсы, но ни тот, ни другой не смогут их получить, так как они уже получили замки. В этом случае программа зависнет.

Пример:

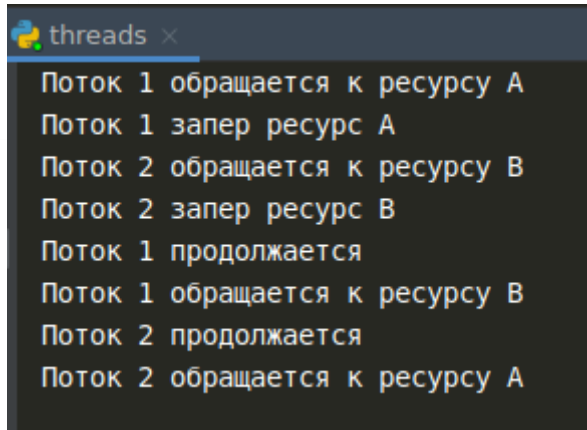
```
import threading
import time

res = {'A': threading.Lock(), 'B': threading.Lock()}

def proc(n, rs):
    for r in rs:
        print("Поток", n, "обращается к ресурсу", r)
        # запрос на запирание замка
        res[r].acquire()
        print("Поток", n, "запер ресурс", r)
        time.sleep(3)
        print("Поток", n, "продолжается")

    for r in rs:
        # запрос на отпирание замка
        res[r].release()
        print("Поток", n, "завершен")

p1 = threading.Thread(target=proc, args=[1, "AB"])
p1.start()
p2 = threading.Thread(target=proc, args=[2, "BA"])
p2.start()
```



```
threads x
Поток 1 обращается к ресурсу A
Поток 1 запер ресурс A
Поток 2 обращается к ресурсу B
Поток 2 запер ресурс B
Поток 1 продолжается
Поток 1 обращается к ресурсу B
Поток 2 продолжается
Поток 2 обращается к ресурсу A
```

Программа зависла из-за взаимных блокировок. Если вставить задержку между потоками, то первый поток успеет закрыть замок, и deadlock не возникнет.

```
p1 = threading.Thread(target=proc, args=[1, "AB"])
p1.start()
time.sleep(5)
p2 = threading.Thread(target=proc, args=[2, "BA"])
p2.start()
```